

How the Attacker Reconstructs the Key from Statistical Signals

TID v1 Case — A Plain-Language Explanation

Author	Ahmad Qasim Mohammad Hassan
Topic	Statistical Flush+Reload Attack on TID v1
Target Audience	Security Researchers — Systems Developers — Report Readers

Part 1 — How AES Encryption Works (Simply)

An AES encryption program has a table in memory called the S-Box — containing 256 slots numbered 0 through 255.
When encrypting any message, the program opens specific slots in the table based on the secret key value.

 **Core Rule**
Different key → Different slots are opened
Fixed key → The same slots are always opened
This is the entry point for the attack.

Part 2 — What the Attacker Sees in TID v1

The attacker monitors all 256 table slots by measuring the cache after every encryption operation.

Slot	Status	Cycles	Meaning
42	Present in cache	78 cycles	Was accessed
87	Present in cache	78 cycles	Was accessed
133	Present in cache	78 cycles	Was accessed
13	Absent	286 cycles	Not used
200	Absent	286 cycles	Not used

In TID v1 this is possible because:

- ✓ Data is wiped from memory
- ✗ But the cache remains loaded
- Attacker reads the cache immediately after the wipe



Part 3 — Statistical Accumulation (The Heart of the Attack)

The attacker repeats the measurement 1,000 times and records how many times each slot appeared.

Slot	Appeared / 1000	Interpretation	Verdict
42	987 times	Always used	⚠ Danger
87	991 times	Always used	⚠ Danger

133	978 times	Always used	⚠ Danger
13	3 times	Rarely used	✓ Safe
200	7 times	Rarely used	✓ Safe

💡 Clear Finding

The key always uses slots 42, 87, and 133 — this reveals the value of parts of the secret key.

Part 4 — How This Becomes a Complete Key

AES-128 key = 16 bytes = 128 binary digits.

🎲 Random Guessing is Impossible

Random guessing: number of possibilities = 2^{128} ← impossible

With the statistical attack, the key is broken byte by byte:

Byte 1: only 256 possibilities → ~1,000 measurements suffice
 Byte 2: only 256 possibilities → ~1,000 additional measurements
 Byte 3: only 256 possibilities → ~1,000 additional measurements
 ... and so on for all 16 bytes

✓ The Statistical Solution

Grand total: $16 \times 1,000 =$ only 16,000 measurements

16,000 measurements instead of 2^{128} — that is what turned the impossible into the possible.

Part 5 — Timeline of the Full Attack

Step	Time	Action
①	Minute 1	Attacker studies the program and learns S-Box addresses
②	Minutes 2–3	Collects 16,000 measurements — each = 112 ns → under 2 seconds actual measurement time
③	Minute 4	Analyses statistics and identifies the most frequently appearing slots
④	Minute 5	Computes every key byte value → full key in hand

Part 6 — Why TID v2 Stops This Completely

⚠ TID v1 — Cache Exposed

After every encryption → cache loaded with the pattern

Attacker measures → sees the pattern

Repeats 16,000 times → builds the key

Pattern + Pattern + Pattern $\times$ 16,000 = Key Exposed ✗

✓ TID v2 — Cache Always Empty

After every encryption → cache fully flushed (CLFLUSHOPT)

Attacker measures → finds silence (286 cycles)

Repeats 16,000 times → silence every time

Zero + Zero + Zero × 16,000 = Nothing ✓

Part 7 — Real Numbers from the Experiment

Scenario	Cycles	Meaning
v1 (no protection)	78 cycles	Clear pattern ← Danger
v2 (with protection)	286 cycles	Silence ← Safe

The 3.7× ratio = the difference between a pattern that can be read and one that vanishes.

The increase from 78 to 286 cycles means the cache is empty and the attacker cannot distinguish between used and unused slots.

Conclusion

- ① Attacker knows the encryption program structure in advance.
- ② Monitors the cache after every encryption operation.
- ③ Records which slots appear consistently.
- ④ Repeated slots reveal the key value.
- ⑤ After 16,000 measurements → full key is known.

TID v2 breaks the chain at step ②:

Cache always empty → No slots appear → No pattern → No key.

— End of Explanation —

Ahmad Qasim Mohammad Hassan | ORCID: 0009-0001-4360-0802 | TID Project — 2026